HTML&CSS. УРОК №4. КЛАССЫ И ИДЕНТИФИКАТОРЫ В HTML И CSS. ТЕГ

Учебные материалы

Урок №4. Классы и идентификаторы в HTML и CSS. Тег

1. Классы и идентификаторы
2. Структура сайта. Создание блочных элементов
3. Блочная верстка

Классы и идентификаторы

Проект на CodePen: https://codepen.io/Academy-top/pen/jOQrJgb?editors=1100

Стилизация и оформление страницы — это важная составляющая в разработке. Преобразить веб-страницу можно с помощью таблиц стилей CSS. При создании различных проектов используют классы и идентификаторы, их также называют селекторами. *Селектор* — это элемент или группа элементов, к которым применяется стиль.

Селекторы незаменимы в тех случаях, когда разработчику необходимо по-особенному оформить страницу или ее элементы, например, изображение, текст или блок текста. То есть, задать особый стиль отдельным элементам. Для задания классов в таблице стилей перед именем необходимо поставить точку, а для ID — решетку (#).

Непосредственно в коде HTML внутри тега, к которому будем применять селектор, добавим class=»» или id=»», а внутри кавычек следует указать уникальное имя (рис. 1)

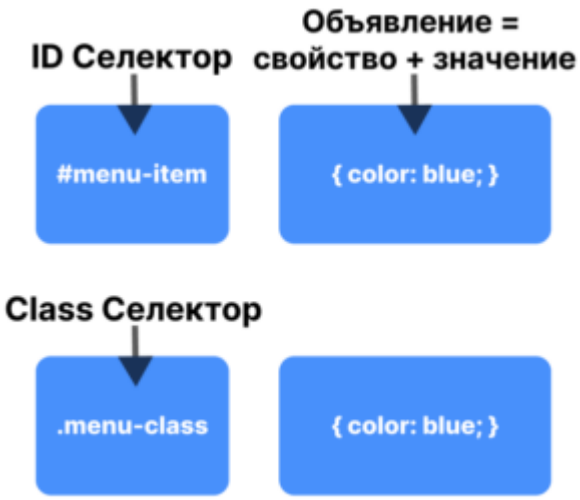


Рисунок 1

Каждый из селекторов имеет свои особенности. Общим для них является то, что и классы, и идентификаторы чувствительны к регистру. Соответственно, нужно следить за тем, буквы какого регистра вводим.

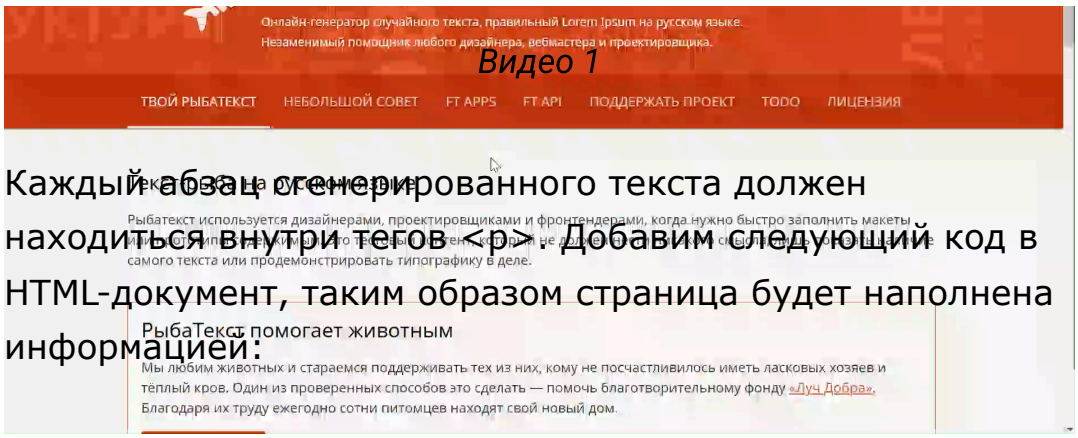
Рассмотрим отличия. Идентификаторы должны быть уникальными и встречаться в коде только один раз. Также они имеют приоритет перед классами. В свою очередь, классы можно применять неоднократно и комбинировать между собой. При сочетании разных классов, если заданы одинаковые свойства (например, дважды определен цвет фона или размер шрифта), будет применяться то свойство, которое в таблице стилей записано в последнюю очередь.

Таким образом, ID следует использовать в тех случаях, если нужно определить стиль только для одного элемента. Если же нужно задать один стиль нескольким элементам, более целесообразно использовать класс.

Рассмотрим на примере использование селекторов для оформления элементов. Добавим рандомный текст или, как его еще называют, текст-рыбку. Используется для временного наполнения блоков, чтобы увидеть, как в целом будет выглядеть страница в итоговом проекте.

Откроем ссылку: https://fish-text.ru/, курсором обозначим число 3 (количество абзацев), оставим абзац (видео 1). Сервис сгенерирует три абзаца, которые можно будет добавить в код.

00:00 00:10



Видео 1

Каждый абзац сгенерированного текста должен находиться внутри тегов <p>. Добавим следующий код в HTML-документ, таким образом страница будет наполнена информацией:

```
<p class="custom-class">
…
...
</p>
<p id="custom-id">
…
...
</p>
<p class="custom-class">
…
...
</p>
```

Вместо троеточия нужно добавить текст-рыбку. Для первого и последнего абзацев <p> добавим класс class=»custom-class», а для второго — идентификатор id= «custom-id». Так будет более наглядно видно, как работают эти селекторы.

Теперь каждому из элементов следует задать свой стиль. Установим отступы и поля для всех <p>:

```
p {
padding: 15px;
margin: 15px;
}
```

Специально для класса custom-class зададим цвет фона и оформление:

```
.custom-class {
background-color: #42b0ff;
border: 4px double black;
}
```

Аналогично сделаем для идентификатора custom-id,- но стиль задаем другой:

```
#custom-id {
background-color: #42b0ff;
border: 4px outset #afdeff;
}
```

Результат будет выглядеть так же, как показано на рисунке 2.

Рисунок 2

В результате мы рассмотрели, как правильно работать с селекторами, а также научились стилизовать элементы с помощью классов и идентификаторов.

Структура сайта. Создание блочных элементов

Для создания любой страницы разработчику необходимо определенным образом размещать основные блоки сайта:

хедер, меню, основную часть и футер. На рисунке 3 приведены элементы, которые должны находиться на странице.

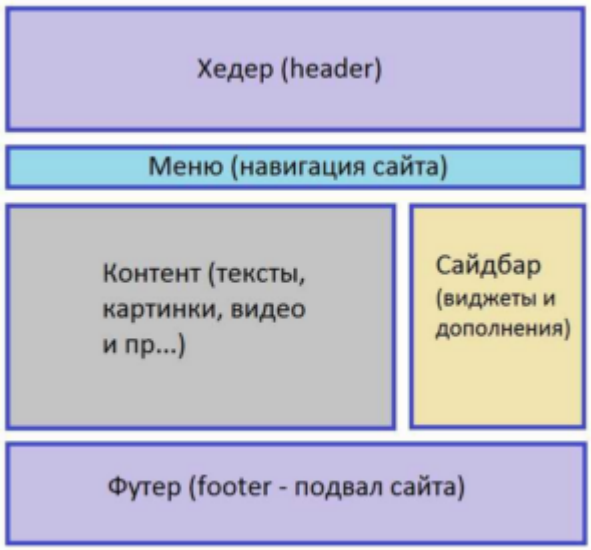


Рисунок 3

Блочная верстка

Ссылка на проект CodePen: https://codepen.io/Academy-top/pen/xxQOeXz?editors=1100

Используя блочную верстку, можно задавать множество параметров. Тег дает возможность создавать блоки и их стиль. Таким образом в одном блоке можно будет объединить огромное количество элементов, например, изображения, текст, заголовки, а также настроить свои

стили. С помощью данного тега удобно создавать адаптивные сайты. Адаптивными называют такие веб-страницы, которые, в зависимости от размера экрана, изменяют размещение содержимого (рис. 4)

Рисунок 4

Создать такую страницу, как уже говорилось, можно с помощью тегов . В блоке HTML следует добавить все необходимые элементы, о которых упоминалось ранее:

```
<div id=”container”>
    <div id="header"><h2>Шапка сайта</h2></div>
    <div id="navigation"><h2>Навигация</h2></div>
    <div id="content"><h2>Контент</h2></div>
    <div id="sidebar"><h2>Сайдбар</h2></div>
    <div id="clear"></div>
    <div id="footer"><h2>Подвал сайта</h2></div>
</div>
```

Теперь же для каждого из элементов зададим стиль. Определим отступы для заголовков второго уровня:

```
h2 {
margin: auto;
}
```

Для каждого элемента div установим отступы для каждой стороны, а также установим позиционирование (расположение) текста:

```
div {
margin-top: 2px;
text-align: center;
}
```

Основной элемент задан с помощью ID с именем container. Для него установим ширину width. Необходимо задать свойство так, чтобы блоки были адаптивны, а, значит, в зависимости от ширины экрана, уменьшались или расширялись. Сделаем это следующим образом:

```
#container {
width: auto;
}
```

Для хедера и навигационного меню установим цвет фона и высоту:

```
#header {
background: #85D6E5;
height: 150px;
}
#navigation {
background: #afdeff;
height: 40px;
}
```

В основной части сайта будет расположен контент (content) и сайдбар (sidebar). Блок content расположим в левой части сайта, занимать он будет большую часть страницы. Выделим для него 75% свободного места.

Указываем значение в процентах, чтобы страница оставалась адаптивной. Вся ширина экрана — это 100%, из них 75% занимает блок content, а значит для sidebar выделяем оставшиеся 25%. В коде со стилями это оформлено следующим образом:

```
#content {
background: #85D6E5;
     float: left;
width: 75%;
height: 250px;
}
#sidebar {
background: #9fd5fc;
     float: right;
width: 25%;
height: 250px;
}
```

Затем идет футер. Наша задача сделать так, чтобы этот элемент находился под content и sidebar. Эту задачу можно решить, задав свойство clear: both перед футером.

```
#clear {
clear: both;
}
#footer {
margin-top: 4px;
background: #75c9d8;
height: 150px;
}
```

В результате блоки будут расположены именно так, как это и предполагалось. При изменении разрешения экрана будет изменяться размер всех блоков (рис. 5).

Рисунок 5

На данный момент мы работали с языком HTML, но следует понимать, что синтаксис разных версий языка отличается. К примеру, если в первом случае мы создавали блоки с помощью тега и соответствующих ID, то в HTML5 их заменяют специальные слова, как в правой части рисунка (рис. 6).

Рисунок 6

Эти теги удобнее использовать, ведь они заменяют целую конструкцию, а для их записи достаточно одного слова.